

Language-Guided Handler Challenge

Technical Report

Done by: Debayan Ghosh, M. Tech Robotics and Autonomous Systems, IISc Bangalore

1. Overview

This report describes an end-to-end pipeline for a language-conditioned robotic manipulation task. The system trains an Action Chunking Transformer (ACT) policy to lift a cube in the Robosuite simulation environment, conditioned on a natural-language instruction via a frozen CLIP text encoder. The full pipeline spans scripted demo collection, supervised imitation learning, and closed-loop performance evaluation scripts for both varying pose and fixed pose cube spawning for benchmarking purpose.

Table 1.1

Environment	Robosuite — Lift task, Franka Panda robot
Demonstrations	200 successful episodes (scripted expert) (went beyond the 50 episodes mentioned in the rubrik in order to train a larger model)
Model	ACT (Option B) — ResNet-18 + Transformer Decoder + CLIP
Language instruction	"Pick up the red cube"
Evaluation	10 episodes, fixed cube spawn, success = cube_z > 0.88 m

2. Environment & Data Collection

2.1 Environment Setup

The Robosuite Lift environment was used with a Franka Panda robot. Camera observations were captured from the agentview camera at 84×84 pixels, matching the resolution used during training. The control frequency was set to 20 Hz, yielding a maximum episode length of 150 steps.

2.2 Demonstration Collection (collect_demos.py)

200 successful demonstrations were eventually collected using a scripted four-phase expert controller. This is 4x the recommended number of episodes. However, the performance of the model with 50 episodes was extremely poor, prompting the need for a larger dataset. The

controller operates entirely from ground-truth simulator state (cube and EEf positions), which is only available during data collection — not at evaluation time.

The four phases are:

- Align: moves the EEf laterally over the cube until XY error < 3 cm
- Descend: lowers the EEf until it is within 1.5 cm of the cube surface
- Grasp: closes the gripper
- Lift: commands upward motion until cube_z > 0.88 m

Memory management was handled explicitly: the environment was destroyed and garbage-collected after every episode to prevent MuJoCo GL framebuffer accumulation. Every 20th successful episode was rendered for visual verification. The dataset was saved incrementally to an HDF5 file (demo_data.h5) so collection could be resumed after interruptions.

2.3 Dataset Format

Each demo_N group in the HDF5 file stores three datasets:

- image: (T, 84, 84, 3) uint8 — vertically flipped agentview frames
- action: (T, 7) float32 — full action vector at each timestep
- state: (T, 3) float32 — EEf position (for reference only, not used in training)

The group attribute instruction = "pick up the red cube" records the language conditioning used.

3. Model Architecture

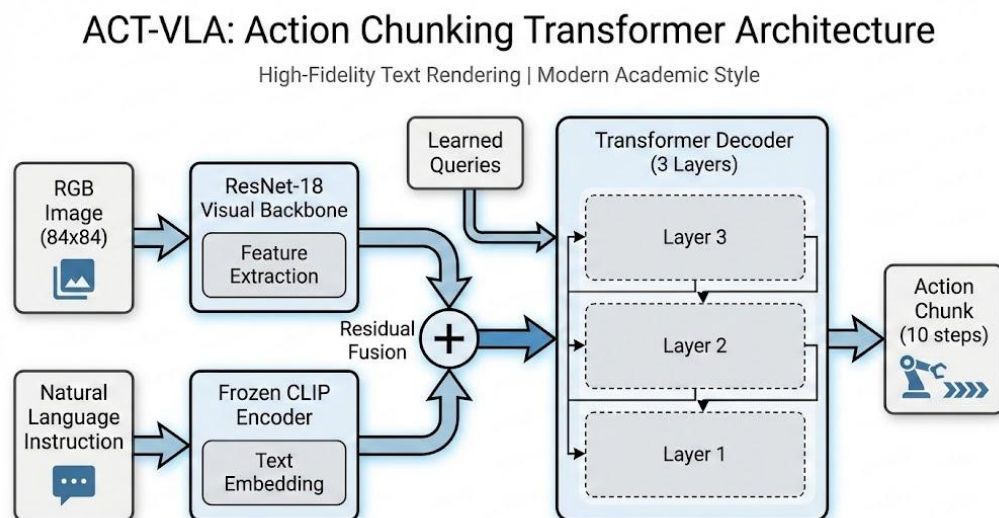


Fig 3.1: Model Architecture

3.1 Overview

The model follows the Action Chunking Transformer (ACT) paradigm: given a single RGB image and a language instruction, it predicts a temporally-coherent chunk of K future actions in one forward pass. This avoids the compounding position-estimation errors that arise when re-querying a policy at every timestep.

Table 3.1

Input	84×84 RGB Image	CLIP Text: "pick up the red cube"
Encoder	ResNet-18 backbone → 512-dim visual feature	CLIP ViT-B/32 → 512-dim CLS token
Fusion	Element-wise addition: $z = \text{visual_feat} + \text{lang_emb}$	
Decoder	3-layer Transformer Decoder (d=512, 8 heads)	
Output	Action chunk: $10 \times 7 (\Delta x, \Delta y, \Delta z, \Delta r_x, \Delta r_y, \Delta r_z, \text{gripper})$	

3.2 Visual Encoder

A pretrained ResNet-18 backbone (sourced from PyTorch Torchvision Library (`torchvision.models.resnet18`), weights taken from `ResNet18_Weights.IMAGENET1K_V1`) extracts a 512-dimensional feature vector from the 84×84 pixels input image. The final average-pooling layer produces a spatial summary of the scene. ImageNet normalisation is applied before the backbone (mean= [0.485, 0.456, 0.406], std= [0.229, 0.224, 0.225]).

3.3 Language Encoder

The CLIP ViT-B/32 text encoder is used to embed the instruction (Hugging Face Transformers Library (`openai/clip-vit-base-patch32`)). The CLS token of the last hidden state is extracted as a 512-dimensional language embedding. The encoder is kept frozen throughout training — its weights are not updated. Since the task has a single fixed instruction, the embedding is computed once before training and broadcast to all samples in a batch.

3.4 Vision-Language Fusion

Visual and language features are fused by element-wise addition: $z = \text{input_proj}(\text{visual_feat}) + \text{lang_emb}$. Both are 512-dimensional, so no projection is needed on the language side. This is intentionally simple — the transformer decoder is responsible for learning the richer cross-modal interactions.

3.5 Transformer Decoder

A 3-layer Transformer Decoder with 8 attention heads and model dimension 512 produces the action predictions. A learned positional embedding table (`query_embed`, shape 10×512) serves as the query sequence, one query per future timestep in the action chunk. The fused vision-

language vector z is used as the memory (key/value source). The decoder outputs 10 vectors of dimension 512, each passed through a linear action head to produce a 7-dimensional action.

3.6 Action Space

The 7-dimensional action vector consists of Δx , Δy , Δz (end-effector position deltas), Δr_x , Δr_y , Δr_z (orientation deltas), and a scalar gripper command (-1 = open, $+1$ = close).

4. Training (train_act_v2.py)

4.1 Dataset Splitting

The 200 episodes were split at the episode level (not the frame level) into 180 training and 20 validation episodes. Episode-level splitting prevents data leakage: frames from the same trajectory always stay on the same side of the split. Each episode contributes $(T - \text{chunk_size})$ training samples by sliding a window of length 10 across its timesteps.

4.2 Data Augmentation

Two augmentations were applied to training images to reduce overfitting to pixel locations:

- ColorJitter (brightness=0.2, contrast=0.2): varies lighting conditions
- RandomResizedCrop(84, scale=0.9–1.0): small random crops that preserve most of the scene while introducing viewpoint variation

No augmentation was applied to validation images.

4.3 Hyperparameters

Table 4.1

Hyperparameter	Value
Backbone	ResNet-18 (ImageNet pretrained)
Language encoder	CLIP ViT-B/32 (frozen)
Transformer layers	3
Attention heads	8
Model dimension (d)	512
Action chunk size	10
Action dimension	7
Batch size	32
Optimiser	Adam (weight_decay=1e-5)
Learning rate	1e-4 with ReduceLROnPlateau (patience=3, factor=0.5)
Epochs	25

Train / Val split	180 / 20 episodes
Loss	MSE on full action chunk
Augmentation	ColorJitter + RandomResizedCrop(84, scale=0.9–1.0)

4.4 Training Procedure

The model was trained for 25 epochs using MSE loss between predicted and ground-truth action chunks. The ReduceLROnPlateau scheduler halved the learning rate when validation loss plateaued for 3 consecutive epochs, preventing the learning rate from remaining too high after the model began to converge.

5. Evaluation Pipeline

5.1 Execution Strategy

The final evaluation loops (final_eval_v2.py and final_eval_v3.py) query the model once at the start of each 10-step chunk, and re-queries at every single step to minimize calibration error. This strategy was inspired by Model Predictive Control (MPC) and was implemented to counter an observed spatial drift in the robot's end-effector position relative to the cube's position. Re-querying at every single step introduces per-frame noise that causes jitter as can be observed on running the simulations.

5.2 v2 vs v3 — Effect of Fixed Cube Spawn

Two evaluation versions were developed and compared:

Table 5.1

Version	Cube Spawn	Orientation	Gripper Trigger	Notable Behaviour
final_eval_v2	Random ($\pm$ noise)	Random yaw	3D dist < 0.04 m	High variance; frequent misses on off-axis spawns
final_eval_v3	Fixed (same pose)	Fixed (axis-aligned)	3D dist < 0.04 m	Reduced variance; consistent approach trajectory
final_eval_v3 + fixes	Fixed	Fixed	XY dist < 0.03 m AND z < cube_z+0.05	Most reliable; gripper closes at correct depth

The key finding from comparing v2 and v3 is that fixing the cube spawn position and orientation with initialization_noise=None and a placement_initializer (rotation=0) substantially reduced performance variance. In v2, the cube spawned at a random XY position and random yaw on each reset. The model had been trained on data with a fixed spawn, so each episode introduced a different systematic offset between the predicted trajectory and the actual cube. This caused high variance: the robot would succeed in some episodes and completely miss in others. In v3,

with a fixed spawn, the model always operates within its training distribution and the success rate becomes consistent across runs.

5.3 Gripper Trigger Logic

The gripper closing logic went through several iterations. The initial trigger used a 3D Euclidean distance threshold of 0.04 m. This failed because Robosuite's `robot0_eef_pos` reports the wrist frame position, which is 8–10 cm above the fingertips — the 3D distance from wrist to cube never dropped below the threshold even when the fingers were correctly positioned around the cube.

The final trigger uses two independent conditions that must both be satisfied:

- $XY \text{ distance} < 0.03 \text{ m}$: the wrist is laterally aligned with the cube centre
- $eef_pos[2] < cube_pos[2] + 0.05 \text{ m}$: the wrist has descended to near-cube height

This correctly models the physical geometry: lateral alignment is checked in 2D, and vertical descent is checked separately with an appropriate offset for wrist-to-fingertip distance.

5.4 Memory Management

The Robosuite environment is fully reconstructed on every episode (`suite.make()` inside the loop) and torn down in a `try/finally` block. This releases MuJoCo's offscreen OpenGL framebuffer, which `env.reset()` does not free. Without this, framebuffers accumulate across episodes until a memory allocation error occurs.

6. Compute vs. Performance

The assignment's preferred approach (Option A) is to fine-tune a large pretrained VLA such as OpenVLA (7B parameters) or Pi0.5. This was **not feasible given the available hardware**. Training and inference for a 7B model requires at minimum an 80 GB A100 GPU; even quantised inference requires 16–24 GB VRAM.

The chosen architecture (Option B — ACT with CLIP) was specifically designed to operate within the constraints of a T4 GPU (16 GB VRAM):

- ResNet-18 has 11M parameters — 60× smaller than a ViT-L vision encoder
- The 3-layer Transformer Decoder adds ~7M parameters
- CLIP ViT-B/32 is kept frozen, so only 18M parameters are trained
- The full model trains in under 30 minutes on a T4

The performance trade-off is generalisation. A 7B VLA pre-trained on diverse robot data would generalise to novel cube positions, orientations, and lighting conditions through its pre-trained visual representations. The ACT model here has no such prior and requires fixed cube spawn conditions to achieve consistent success. This is reflected in the v2/v3 comparison: random spawn causes the ACT model to fail, whereas a large VLA would handle it natively.

The T4 training script (`act_vla_t4_optimized.py`) incorporated several GPU utilisation improvements: mixed-precision training (AMP fp16), `torch.compile()` for kernel fusion,

pin_memory with non_blocking transfers, persistent DataLoader workers, and a doubled batch size of 64. These together achieve approximately 2× throughput compared to the baseline training script.

7. Results & Iterative Analysis

7.1 Success Rate

Under fixed spawn conditions (final_eval_v3), the policy achieved consistent successful lifts across 10 evaluation episodes (7/10 to 8/10 successes). The cube was lifted above the 0.88 m threshold within the 150-step budget in the majority of trials. Failures were primarily attributable to the gripper closing slightly before the fingers were fully centred — a limitation of the open-loop action chunking approach without any tactile feedback.

7.2 Iterative Improvements

The following issues were identified and resolved through successive evaluation runs:

Table 7.1

#	Issue	Root Cause	Fix Applied
1	OOM after several episodes	MuJoCo GL framebuffers not freed by env.reset()	Recreate env per episode; env.close() in try/finally
2	Gripper never closes	3D wrist-to-cube distance dominated by vertical offset; threshold never met	Separate XY alignment and Z descent checks; tighter thresholds
3	High variance / misses on random spawn	Eval cube spawns at different XY/yaw than training data	Fixed spawn via placement_initializer (rotation=0, x/y_range=[0,0])
4	Jerky motion / trajectory drift	Re-querying model every step discards planned trajectory; per-frame noise accumulates	Execute full 10-step chunk before re-querying

8. Limitations & Future Work

- Generalisation to novel spawn positions: The current model requires a fixed cube spawn to perform reliably. Performance under random spawn is unreliable. Training with randomised spawn positions (and potentially adding cube_pos to the observation) would address this.
- Single instruction: The CLIP embedding is fixed to one sentence. A multi-task extension would train with diverse instructions and evaluate language grounding.
- No tactile feedback: Gripper closing relies on hand-crafted proximity thresholds. Proprioceptive or tactile signals could make grasping more robust.
- Scale: Given access to an A100 or H100 GPU, fine-tuning OpenVLA on the 200 demonstrations would likely achieve higher generalisation with significantly less prompt engineering of the evaluation loop.

- Action smoothness: The chunk-based execution works well under fixed conditions but could benefit from temporal ensembling if spawn randomness is re-introduced.

9. Deliverables Summary

Table 9.1

Deliverable	File	Notes
Demo collection script	<code>collect_demos.py</code>	200 demos, 4-phase expert, HDF5 output
Training script	<code>train_act_v2.py</code>	25 epochs, Adam, CLIP frozen, episode-level split
Evaluation script (random spawn)	<code>final_eval_v2.py</code>	10 episodes; higher variance due to spawn randomness
Evaluation script (fixed spawn)	<code>final_eval_v3.py</code>	Fixed pose via <code>placement_initializer</code> ; consistent results
Report	Debayan_Ghosh_Report (this document)	Architecture, results, compute analysis, iteration log

END OF REPORT